UFUG2106 Discrete Mathematics Final Project

Round 5.1 —WSL Local Reproduction & Generalization Audit

Experiment timestamp: 2026-11-11 13:45:47 (UTC+08:00)
Source archive: `20261111_134547_Round5.1_local_results.zip`
System: Windows Subsystem for Linux 2; Linux kernel 6.6.87.2; Ubuntu GCC 13.3.0; Python 3.12.3.
Comparison: Frozen Round 5 (reference) versus Round 5.1 (typed in-memory DAG implementation).

Abstract

Round 5.1 replaces repeated text-oriented netlist transformations with a typed, in-memory directed acyclic graph for key postprocessing stages and propagates gate arrival times upon node construction. The Boolean synthesis heuristics, candidate scoring rules, and logic identities are inherited from Round 5. This audit compares both binaries freshly compiled on the user's WSL environment. Across two interleaved 25-case Stage 1 paired runs, all 100 evaluator outcomes pass correctness and timing targets. Area remains 529,014 for each run and version; the mean of per-run median runtimes falls from 66.5075 ms to 55.2825 ms (16.88% lower). The first run's 25 final netlists are byte-identical. Extra function corpora show full functional equivalence in evaluated cases but mixed runtime gains, particularly for small circuits. The decision is to accept Round 5.1 as the efficiency baseline, without claiming universal speedups or immunity to hidden-stage overfitting.

1. Motivation and implementation

The previous Round 5 implementation repeatedly converted intermediate netlists between text and gate structures in rebalancing and area recovery. Round 5.1 introduces an in-memory graph

$$G = (V, E, \mathsf{op}, \mathsf{fanin}, \mathsf{outputs}),$$

where every gate node's fanins precede it in topological order. Its typed representation stores gate kind and fanin identifiers, and reuses nodes via structural hashing. For a new gate node v, arrival times satisfy

$$t(v) = \begin{cases} t(a) + d_{\neg}, & v = \mathsf{NOT}(a), \\ \max\{t(a), t(b)\} + d_{\wedge}, & v = \mathsf{AND2}(a, b), \\ \max\{t(a), t(b)\} + d_{\vee}, & v = \mathsf{OR2}(a, b). \end{cases}$$

The recurrence is exact by topological induction, provided gates are immutable and referenced fanins have already been constructed. Algebraic and timing-aware rewrite rules use previously established Boolean identities. Exact total Area is still evaluated after liveness pruning:

$$A(G) = \sum_{v \in V_{\mathsf{live}} \setminus V_{\mathsf{inputs}}} a_{\mathsf{op}(v)}, \quad D(G) = \max_{y \in \mathsf{outputs}} t(y).$$

Scope limitation: This is *not* a fully incremental Area solver. Global sharing and dead-node effects still require exact live-graph accounting. Later algebraic and phase-mapping paths may still use string-based interfaces; the typed graph accelerates specific postprocessing stages, not the entire synthesis pipeline.

2. Local experimental protocol

Both executables are freshly compiled using the same compiler flags and official evaluator. The script runs two *opposite-order* comparisons: Run 1 evaluates Round 5 followed by Round 5.1; Run 2 reverses the order. Both evaluate the same 25 official input cases. Run 1 saves both output netlists for byte

comparison. Functional correctness and delay targets are scored by the official evaluator; the saved Round 5.1 netlists are independently checked (13 exhaustive, 12 deterministic sampled).

Measurement qualification. Two interleaved repeats reduce, but do not eliminate, transient CPU/load/order effects. These are descriptive measurements, not a confidence interval or a claim of universal speedup. The checksum and case-manifest files in the original experiment folder provide reproducibility fingerprints.

3. Official Stage 1 results

Metric	Run 1 Round 5	Run 1 Round 5.1	Run 2 Round 5	Run 2 Round 5.1
Official PASS	25/25	25/25	25/25	25/25
Target Delay Met	25/25	25/25	25/25	25/25
Area sum	529,014	529,014	529,014	529,014
Median Runtime (ms)	72.188	55.488	60.827	55.077
Runtime sum (ms)	7,866.005	6,517.229	6,648.879	5,877.434
Cases faster in Round 5.1	—	23/25	—	21/25

Mean-of-medians improvement:

$$1 - \frac{(55.488 + 55.077)/2}{(72.188 + 60.827)/2} = 16.88\%.$$

The *total runtime* over both 25-case runs decreases from 14,514.884 ms to 12,394.663 ms, a 14.61% reduction. Round 5.1's per-case runtime did not always improve: in Run 1, case_024 and case_020 were slower; in Run 2, four cases were slower. This reflects the noise and uneven amortization of the typed-graph machinery, not failures in circuit equivalence. Full 50-pair data are archived in `Round5.1_PerCase_Paired_Comparison.csv`.

The first pair's 25 netlists were independently byte-compared and found **25/25 identical**. Every case's Area and Delay matches in both benchmark runs. The standalone netlist validator recorded **25/25 matches**, with 13 exhaustive proofs by enumeration of the finite input space and 12 deterministic-sample checks. The official evaluator separately reports 25/25 functional PASS per run.

4. Out-of-sample and regression experiments

The archive includes four additional corpora, tested on both frozen Round 5 and Round 5.1. Their measured results on this WSL machine are:

Corpus	Cases	PASS (both)	Timing targets (both)	Area / Delay match	R5 median (ms)	R5.1 median (ms)
Mixed structured holdout	80	80/80	80/80	80/80	7.595	6.932
Larger structured holdout	16	16/16	16/16	16/16	76.995	70.605
Earlier generated tests	110	110/110	96/110	110/110	5.034	5.175
Stress tests	2	2/2	2/2	2/2	1,023.142	801.408

Important negative result: in the older 110-case corpus, Round 5.1's median runtime is **2.8% higher** than Round 5. The 80-case and 16-case new holdouts each show roughly 8–9% lower median runtimes; both are smaller improvements than the originally reported container results. The two stress tests have too few observations for a stable median inference. Correctness and Area/Delay equality hold on every tested case; the 14 target misses among older generated functions are shared by *both* versions and should not be conflated with functional failures.

Overfitting assessment. Preserving the candidate generation logic and evaluating additional functions limits the risk of direct tuning to the 25 public cases. Still, the new holdouts are synthetic and may share design assumptions with the algorithm. This experiment cannot establish generalization to the unreleased Stage 2 distribution; nor can it settle whether the earlier variable-order and area heuristics are overfit.

5. Reproducibility evidence and caveats

- Source and evaluator SHA-256 hashes are preserved in `environment.txt`; the original archive also contains case and holdout hashes.
- Full official CSVs, console logs, and all 25 saved netlists for both Run 1 executables are retained in the original uploaded ZIP.
- The experiment uses interleaved opposite-order execution, but only one WSL machine and two repetitions. Variability across runs is non-negligible.
- A time-budgeted portfolio may choose different valid candidates if optimization changes how many candidates finish before a deadline. Byte equality in one run is empirical, not a universal semantic theorem.
- Stage 2 inputs are unavailable, so claims about future hidden-test behavior remain hypotheses.

6. Decision and next-step gate

Accept Round 5.1 as a frozen efficiency baseline for further research, with Round 5 retained as a rollback and comparison branch. The combination of (a) 25/25 official correctness + target hits in both runs, (b) identical cost metrics, (c) 25/25 byte equality in the saved pair, and (d) no held-out functional regressions is sufficient for this project-level engineering decision.

Do not pursue further micro-optimization simply to improve the Stage 1 median. The current measured gain is useful but modest and input-size dependent. If there is another research round, spend it on delay-constrained Area optimization (e.g. cut-based local rewriting) and evaluate new candidates against untouched, diverse, *separately recorded* tests. Use an explicit acceptance rule: never trade functional correctness or target satisfaction for area gains; measure performance on both public and fresh held-out families, not just Stage 1 aggregate area.

7. File layout and citations within the archive

- `Round5.1_PerCase_Paired_Comparison.csv`: two full 25-case pairing tables.
- `Round5.1_Holdout_Comparison.csv`: 208 additional case comparisons.
- `Round5.1_Local_Statistics.json`: computed summaries and validation assertions.
- `Round5.1_Report_Ready_README.md`: placement instructions.
- Original upload: `20261111_134547_Round5.1_local_results.zip` contains all underlying source snapshots, evaluator outputs, logs, and checksums.